DATABASE DESIGN DOCUMENT

Autonomous Video Ad-Placement & Contextual Safety Analyzer

Submitted in partial fulfillment of the requirements for the course

Specialization Project

Submitted By

Andrew Shelton S	2441609
Andrew Judah	2441608
Akash Ganesh	2441604
Ramona Elisha Mary	2441646

Under the Guidance of

[Guide Name]

Assistant Professor

Department of Computer Science

School of Sciences

CHRIST (Deemed to be University)

Bangalore

Academic Year 2026–2027

Contents

1 Introduction **4**

1.1 Purpose 4

1.2 Scope 4

1.3 Intended Audience 4

1.4 Definitions and Acronyms 4

2 Database Requirements **5**

2.1 Functional Data Requirements 5

2.2 Non-Functional Data Requirements 6

3 Entity Identification **6**

4 Entity Relationship Diagram **7**

4.1 Relationship Cardinality Analysis 7

5 Relational Schema **8**

5.1 VideoMetadata Schema 8

5.2 AudioFlags Schema 9

5.3 VisualFlags Schema 9

5.4 AdSpots Schema 10

5.5 AuditReports Schema 10

5.6 SystemConfig Schema 10

6 Data Dictionary **11**

6.1 VideoMetadata Table 11

6.2 AudioFlags Table 12

6.3 VisualFlags Table 12

6.4 AdSpots Table 13

6.5 AuditReports Table 13

6.6 SystemConfig Table 14

7 Normalization **14**

7.1 First Normal Form (1NF) 14

7.2 Second Normal Form (2NF) 15
7.3 Third Normal Form (3NF) 15

8 SQL Table Definitions **15**
8.1 Database Initialization & Pragmas 15
8.2 VideoMetadata Table Definition 16
8.3 AudioFlags Table Definition 16
8.4 VisualFlags Table Definition 17
8.5 AdSpots Table Definition 17
8.6 AuditReports Table Definition 18
8.7 SystemConfig Table Definition 18

9 Sample Records **19**
9.1 VideoMetadata Table 19
9.2 AudioFlags Table 19
9.3 VisualFlags Table 20
9.4 AdSpots Table 20

10 Database Architecture **20**
10.1 Architectural Layer Integration 20

11 Conclusion **22**

12 References **22**

1. Introduction

1.1 Purpose

The purpose of this Database Design Document (DDD) is to provide a comprehensive architectural blueprint for the database layer of the **Autonomous Video Ad-Placement & Contextual Safety Analyzer**. This document details the data models, entity relationships, relational schemas, normalization principles, data dictionaries, and SQL DDL specifications required to store and manage video metadata, multimodal AI audit flags, ad-spot spatial coordinates, and generated compliance reports.

1.2 Scope

The application is designed as a localized, offline desktop system constructed using Python and Streamlit. To ensure complete data privacy, zero cloud latency, and zero API costs, the system utilizes an embedded relational database engine (**SQLite3**). The database layer acts as the central orchestrator between parallel AI processing engines:

- **Audio & NLP Engine (OpenAI Whisper):** Stores timestamped speech transcripts, flagged profanity, and compliance severity ratings.
- **Computer Vision Engine (Ultralytics YOLOv8 & OpenCV):** Records sampled frame timestamps, detected visual risks (e.g., weapons, smoking, alcohol), bounding box coordinates, and continuous planar ad-spot geometry.
- **Dashboard & Reporting UI:** Dynamically queries stored records to render interactive timelines, filter audit events, and export formal PDF/CSV reports.

1.3 Intended Audience

This document is intended for software architects, database administrators, backend developers, AI pipeline engineers, quality assurance testers, and project evaluators.

1.4 Definitions and Acronyms

- **DDD** – Database Design Document

- **ERD** – Entity Relationship Diagram
- **DDL** – Data Definition Language
- **PK** – Primary Key
- **FK** – Foreign Key
- **1NF / 2NF / 3NF** – First, Second, and Third Normal Forms
- **ACID** – Atomicity, Consistency, Isolation, Durability
- **JSON** – JavaScript Object Notation (used for storing bounding box coordinates)

2. Database Requirements

The database subsystem is designed to satisfy rigorous functional and non-functional requirements tailored for offline video processing:

2.1 Functional Data Requirements

- **Video Metadata Management:** Capture and store comprehensive video file attributes including filename, local storage path, total duration, frame rate (FPS), resolution, and real-time processing status.
- **Audio Compliance Logging:** Record timestamped audio segments transcribed by Whisper, store detected keyword violations, categorize risk severity (Low, Medium, High), and track specific dictionary matches.
- **Visual Hazard Tracking:** Store object detection logs generated by YOLOv8, including exact frame timestamps, detected class labels (e.g., firearm, alcohol, illicit substance), bounding box coordinates, and model confidence scores.
- **Ad-Placement Geometry Storage:** Record spatial dimensions of identified prime ad-placement surfaces (e.g., blank walls, laptop screens, empty desks), including start/end timestamps, surface classification, planar pixel area, and bounding box arrays.

- **Audit Report Archiving:** Maintain historical records of generated compliance audit summaries and exported reports.

- **System Configuration Storage:** Store user-defined dictionary rules, risk tolerance thresholds, and model inference parameters.

2.2 Non-Functional Data Requirements

- **Offline Data Privacy:** All data must be persisted locally on the host machine using SQLite3, ensuring no proprietary video or audio data is ever transmitted over the network.

- **High-Speed Timestamp Querying:** Support rapid range queries on timestamps to allow seamless synchronization between video playback on the dashboard and flagged compliance events.

- **Concurrency & Transaction Safety:** Ensure ACID compliance and thread-safe database connections to handle concurrent write operations from parallel audio and video processing threads.

- **Data Integrity:** Enforce strict referential integrity using foreign key constraints and data validation rules.

3. Entity Identification

Based on the system requirements, six core entities have been identified to model the domain.

Entity	Description
VideoMetadata	Central entity storing uploaded video file properties, ingestion timestamps, and pipeline processing status.
AudioFlags	Stores timestamped speech violations, transcribed text segments, flagged profanity words, and risk severity ratings.
VisualFlags	Stores frame-level visual hazard detections, object class labels, bounding box coordinates, and YOLOv8 confidence scores.
AdSpots	Stores spatial geometry and temporal durations of identified non-intrusive sponsor ad-placement surfaces.
AuditReports	Records metadata and summary statistics for exported compliance reports (PDF/CSV generated by auditors).
SystemConfig	Stores customizable application settings, profanity dictionary terms, and detection sensitivity thresholds.

4. Entity Relationship Diagram

The Entity Relationship Diagram (ERD) illustrates the conceptual schema and cardinality between entities within the system.

4.1 Relationship Cardinality Analysis

- **VideoMetadata to AudioFlags (1 : N):** A single video can contain zero, one, or multiple audio compliance flags across its timeline. Each audio flag maps to exactly one video.

- **VideoMetadata to VisualFlags (1 : N):** A single video can generate multiple visual hazard detections across sampled frames. Each visual flag belongs to exactly

one video.

- **VideoMetadata to AdSpots (1 : N):** A single video can feature multiple continuous ad-placement opportunities. Each ad spot is tied to one specific video.
- **VideoMetadata to AuditReports (1 : N):** A user can generate multiple compliance reports (e.g., preliminary vs. final audits) for a single video.
- **SystemConfig (Independent / Singleton):** Maintains global application parameters and custom dictionary rules referenced by the AI processing engines.



Figure 1: Entity Relationship Diagram (ERD) of the System

5. Relational Schema

The conceptual ERD is mapped into formal relational database schemas. Primary Keys (PK) and Foreign Keys (FK) are explicitly defined to enforce referential integrity.

5.1 VideoMetadata Schema

```
VideoMetadata (
```

```
    video_id PK,
    filename,
    filepath,
    file_size_mb,
    duration_sec,
    fps,
    resolution_width,
    resolution_height,
    upload_timestamp,
    processed_status
)
```

5.2 AudioFlags Schema

```
AudioFlags (
    audio_flag_id PK,
    video_id FK,
    timestamp_start,
    timestamp_end,
    word_flagged,
    transcript_context,
    severity,
    detection_model
)
```

5.3 VisualFlags Schema

```
VisualFlags (
    visual_flag_id PK,
    video_id FK,
    frame_timestamp,
    object_detected,
    bounding_box_json,
    confidence_score,
```

```
    risk_category
)
```

5.4 AdSpots Schema

```
AdSpots (
    spot_id PK,
    video_id FK,
    timestamp_start,
    timestamp_end,
    spot_type,
    bounding_box_json,
    area_pixels,
    suitability_score
)
```

5.5 AuditReports Schema

```
AuditReports (
    report_id PK,
    video_id FK,
    generation_timestamp,
    report_type,
    file_path,
    total_audio_risks,
    total_visual_risks,
    total_ad_spots,
    compliance_status
)
```

5.6 SystemConfig Schema

```
SystemConfig (
    config_key PK,
```

```
    config_value,
    config_type,
    description,
    last_updated
)
```

6. Data Dictionary

The Data Dictionary provides detailed attribute specifications, data types, storage sizes, and constraints for all relational tables in the SQLite3 database.

6.1 VideoMetadata Table

Attribute	**Type**	**Size**	**Constraint / Description**
video_id	INTEGER	8	Primary Key, Auto-increment
filename	TEXT	255	Not Null, Original filename
filepath	TEXT	500	Not Null, Local disk storage path
file_size_mb	REAL	8	Not Null, Size in megabytes
duration_sec	REAL	8	Not Null, Total video duration
fps	REAL	8	Not Null, Frames per second
resolution_width	INTEGER	4	Width in pixels (e.g., 1920)
resolution_height	INTEGER	4	Height in pixels (e.g., 1080)
upload_timestamp	TEXT	30	Default CURRENT_TIMESTAMP
processed_status	TEXT	20	CHECK (pending, processing, completed, failed)

6.2 AudioFlags Table

Attribute	Type	Size	Constraint / Description
audio_flag_id	INTEGER	8	Primary Key, Auto-increment
video_id	INTEGER	8	Foreign Key → VideoMetadata
timestamp_start	REAL	8	Not Null, Start time in seconds
timestamp_end	REAL	8	Not Null, End time in seconds
word_flagged	TEXT	100	Not Null, Restricted word detected
transcript_context	TEXT	500	Surrounding sentence text
severity	TEXT	20	CHECK (Low, Medium, High)
detection_model	TEXT	50	Default 'Whisper-base.en'

6.3 VisualFlags Table

Attribute	Type	Size	Constraint / Description
visual_flag_id	INTEGER	8	Primary Key, Auto-increment
video_id	INTEGER	8	Foreign Key → VideoMetadata
frame_timestamp	REAL	8	Not Null, Timestamp in seconds
object_detected	TEXT	100	Not Null, YOLO class label
bounding_box_json	TEXT	200	Not Null, JSON array [x1,y1,x2,y2]
confidence_score	REAL	8	Not Null, Score between 0.0 and 1.0
risk_category	TEXT	50	e.g., Weapon, Substance, Graphic

6.4 AdSpots Table

Attribute	Type	Size	Constraint / Description
spot_id	INTEGER	8	Primary Key, Auto-increment
video_id	INTEGER	8	Foreign Key → VideoMetadata
timestamp_start	REAL	8	Not Null, Start time in seconds
timestamp_end	REAL	8	Not Null, End time in seconds
spot_type	TEXT	50	e.g., Blank Wall, Laptop Screen
bounding_box_json	TEXT	200	Not Null, JSON array [x1,y1,x2,y2]
area_pixels	REAL	8	Continuous surface area
suitability_score	REAL	8	Score based on stability/size

6.5 AuditReports Table

Attribute	Type	Size	Constraint / Description
report_id	INTEGER	8	Primary Key, Auto-increment
video_id	INTEGER	8	Foreign Key → VideoMetadata
generation_timestamp	TEXT	30	Default CURRENT_TIMESTAMP
report_type	TEXT	10	CHECK (PDF, CSV, JSON)
file_path	TEXT	500	Local path to saved report file
total_audio_risks	INTEGER	4	Total audio violations count
total_visual_risks	INTEGER	4	Total visual hazards count
total_ad_spots	INTEGER	4	Total ad opportunities count
compliance_status	TEXT	20	e.g., Monetizable, Restricted

6.6 SystemConfig Table

Attribute	Type	Size	Constraint / Description
config_key	TEXT	100	Primary Key, Unique setting name
config_value	TEXT	500	Not Null, Stored parameter value
config_type	TEXT	20	CHECK (string, int, float, json)
description	TEXT	255	Purpose of configuration item
last_updated	TEXT	30	Default CURRENT_TIMESTAMP

7. Normalization

To eliminate data redundancy, prevent insertion/deletion anomalies, and ensure relational efficiency, the database schema has been normalized up to the **Third Normal Form (3NF)**.

7.1 First Normal Form (1NF)

A relational table satisfies 1NF if it contains only atomic values and no repeating groups.

- **Implementation:** In our initial conceptual design, visual detections and ad spots could have been stored as repeating groups or nested lists within the video metadata record. To achieve 1NF, we decomposed these into dedicated child tables (`AudioFlags`, `VisualFlags`, and `AdSpots`).

- **Atomicity:** Complex spatial coordinates (bounding boxes) are stored as standardized JSON strings representing atomic arrays `[x1, y1, x2, y2]`, ensuring each table attribute evaluates to a single scalar value per record.

7.2 Second Normal Form (2NF)

A table satisfies 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the primary key (no partial dependencies).

- **Implementation:** All tables in our schema utilize single-attribute surrogate primary keys (`video_id`, `audio_flag_id`, `visual_flag_id`, `spot_id`, `report_id`). Because there are no composite primary keys, partial dependency is mathematically impossible. Every attribute in `AudioFlags` (such as `word_flagged` and `severity`) depends entirely on `audio_flag_id`.

7.3 Third Normal Form (3NF)

A table satisfies 3NF if it is in 2NF and contains no transitive dependencies (non-prime attributes must not depend on other non-prime attributes).

- **Implementation:** We examined relationships between non-prime attributes across all tables. For instance, in an earlier iteration, video filename, resolution, and duration were duplicated inside the audit report table. This created a transitive dependency: `report_id` → `video_id` → `filename`.
- **Resolution:** We removed all duplicated video metadata from `AuditReports`, `AudioFlags`, `VisualFlags`, and `AdSpots`. These child tables now only reference `video_id` as a foreign key. Any required video properties are retrieved via relational `JOIN` queries, guaranteeing zero transitive dependency and strict 3NF compliance.

8. SQL Table Definitions

This section provides the complete Data Definition Language (DDL) scripts required to initialize the SQLite3 database schema, including foreign key constraints, check constraints, and performance indexes.

8.1 Database Initialization & Pragmas

```
-- Enable Foreign Key enforcement in SQLite3
PRAGMA foreign_keys = ON;
```

8.2 VideoMetadata Table Definition

```
CREATE TABLE VideoMetadata (
    video_id INTEGER PRIMARY KEY AUTOINCREMENT,
    filename TEXT NOT NULL,
    filepath TEXT NOT NULL,
    file_size_mb REAL NOT NULL,
    duration_sec REAL NOT NULL,
    fps REAL NOT NULL,
    resolution_width INTEGER DEFAULT 1920,
    resolution_height INTEGER DEFAULT 1080,
    upload_timestamp TEXT DEFAULT CURRENT_TIMESTAMP,
    processed_status TEXT DEFAULT 'pending'
        CHECK (processed_status IN ('pending', 'processing',
                                    'completed', 'failed'))
);
```

8.3 AudioFlags Table Definition

```
CREATE TABLE AudioFlags (
    audio_flag_id INTEGER PRIMARY KEY AUTOINCREMENT,
    video_id INTEGER NOT NULL,
    timestamp_start REAL NOT NULL,
    timestamp_end REAL NOT NULL,
    word_flagged TEXT NOT NULL,
    transcript_context TEXT,
    severity TEXT DEFAULT 'Medium'
        CHECK (severity IN ('Low', 'Medium', 'High')),
    detection_model TEXT DEFAULT 'Whisper-base.en',
    FOREIGN KEY (video_id) REFERENCES VideoMetadata(video_id)
        ON DELETE CASCADE
);
```

```
-- Index for rapid timeline filtering on dashboard
CREATE INDEX idx_audio_video_time
ON AudioFlags(video_id, timestamp_start);
```

8.4 VisualFlags Table Definition

```
CREATE TABLE VisualFlags (
    visual_flag_id INTEGER PRIMARY KEY AUTOINCREMENT,
    video_id INTEGER NOT NULL,
    frame_timestamp REAL NOT NULL,
    object_detected TEXT NOT NULL,
    bounding_box_json TEXT NOT NULL,
    confidence_score REAL NOT NULL CHECK (confidence_score >= 0.0
                                          AND confidence_score <= 1.0),
    risk_category TEXT NOT NULL,
    FOREIGN KEY (video_id) REFERENCES VideoMetadata(video_id)
        ON DELETE CASCADE
);

-- Index for frame timestamp lookups
CREATE INDEX idx_visual_video_time
ON VisualFlags(video_id, frame_timestamp);
```

8.5 AdSpots Table Definition

```
CREATE TABLE AdSpots (
    spot_id INTEGER PRIMARY KEY AUTOINCREMENT,
    video_id INTEGER NOT NULL,
    timestamp_start REAL NOT NULL,
    timestamp_end REAL NOT NULL,
    spot_type TEXT NOT NULL,
    bounding_box_json TEXT NOT NULL,
    area_pixels REAL NOT NULL,
    suitability_score REAL DEFAULT 0.8,
```

```
    FOREIGN KEY (video_id) REFERENCES VideoMetadata(video_id)
        ON DELETE CASCADE
);

-- Index for ad-spot duration queries
CREATE INDEX idx_adspots_video_time
ON AdSpots(video_id, timestamp_start);
```

8.6 AuditReports Table Definition

```
CREATE TABLE AuditReports (
    report_id INTEGER PRIMARY KEY AUTOINCREMENT,
    video_id INTEGER NOT NULL,
    generation_timestamp TEXT DEFAULT CURRENT_TIMESTAMP,
    report_type TEXT NOT NULL CHECK (report_type IN ('PDF', 'CSV', 'JSON')),
    file_path TEXT NOT NULL,
    total_audio_risks INTEGER DEFAULT 0,
    total_visual_risks INTEGER DEFAULT 0,
    total_ad_spots INTEGER DEFAULT 0,
    compliance_status TEXT DEFAULT 'Monetizable',
    FOREIGN KEY (video_id) REFERENCES VideoMetadata(video_id)
        ON DELETE CASCADE
);
```

8.7 SystemConfig Table Definition

```
CREATE TABLE SystemConfig (
    config_key TEXT PRIMARY KEY,
    config_value TEXT NOT NULL,
    config_type TEXT DEFAULT 'string'
        CHECK (config_type IN ('string', 'int', 'float', 'json')),
    description TEXT,
    last_updated TEXT DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Insert Default System Configurations
INSERT INTO SystemConfig (config_key, config_value, config_type, description)
VALUES
('whisper_model_size', 'base.en', 'string', 'Whisper STT model weight size'),
('yolo_confidence_thresh', '0.50', 'float', 'Minimum YOLO detection score'),
('ad_spot_min_area_px', '25000', 'int', 'Minimum planar area for ad overlay'),
('profanity_filter_level', 'strict', 'string', 'Audio risk classification mode');
```

9. Sample Records

To demonstrate database population during a typical video audit session, realistic sample records are presented below.

9.1 VideoMetadata Table

video_id	filename	duration_sec	fps	resolution	status
1	tech_review_ep4.mp4	645.5	30.0	1920x1080	completed
2	gaming_stream_raw.avi	1820.0	60.0	2560x1440	completed
3	podcast_interview.mp4	3400.2	24.0	1920x1080	processing

9.2 AudioFlags Table

id	video_id	start	end	word_flagged	severity
101	1	45.2	46.0	damn	Low
102	1	180.5	181.2	sh*t	Medium
103	2	12.0	13.5	f***	High
104	2	450.8	451.6	b****	Medium

9.3 VisualFlags Table

id	video_id	time	object_detected	bounding_box	conf
201	1	120.0	wine_glass	[450,300,520,480]	0.88
202	2	85.0	firearm_prop	[100,150,350,400]	0.92
203	2	86.0	firearm_prop	[105,152,355,402]	0.91
204	2	910.0	cigarette	[600,400,630,420]	0.79

9.4 AdSpots Table

id	video_id	start	end	spot_type	area_px
301	1	10.0	40.0	blank_wall_background	185000.0
302	1	200.0	260.0	laptop_screen_display	64000.0
303	2	300.0	450.0	empty_desk_surface	95000.0

10. Database Architecture

The database architecture is designed as an embedded, serverless storage layer integrated directly within the Python desktop environment. Unlike traditional client-server RDBMS systems (such as MySQL or PostgreSQL), SQLite3 stores the entire database schema, tables, and indices within a single cross-platform file on the local file system.

10.1 Architectural Layer Integration

1. **Presentation Layer (Streamlit UI):** Connects to SQLite3 via SQLAlchemy or native Python SQLite cursors in read-only mode during timeline scrubbing, querying flagged timestamps instantly to update visual charts and UI components.

2. **Controller & Orchestration Layer:** Manages transaction boundaries, initiating database records upon video upload and handling status updates (`pending` → `processing` → `completed`).

3. **Multimodal AI Worker Threads:**

- **Audio Worker:** Transcribes audio and writes batch insert transactions to `AudioFlags`.
- **Vision Worker:** Samples video frames at 1 FPS, performs YOLOv8 spatial inference, and writes bounding box arrays to `VisualFlags` and `AdSpots`.

4. **Storage Layer (SQLite3 Engine):** Utilizes Write-Ahead Logging (`PRAGMA journal_mode = WAL`) to permit simultaneous read operations from the UI while AI background threads perform database insertions without locking or database contention.

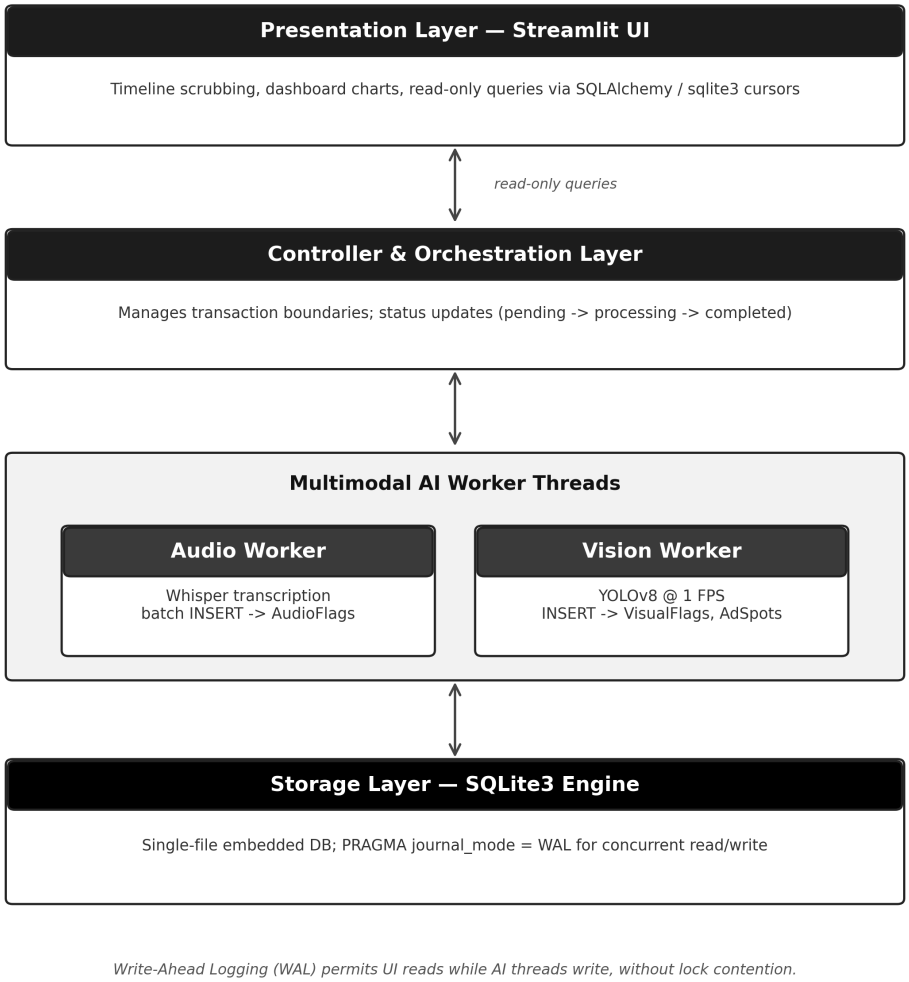


Figure 2: Database Architecture and Multi-Threaded Pipeline Integration

11. Conclusion

The database design specified in this document establishes a robust, highly structured, and normalized data foundation for the **Autonomous Video Ad-Placement & Contextual Safety Analyzer**. By leveraging an embedded SQLite3 architecture with strict 3NF relational schemas, automated indexing, and Write-Ahead Logging, the system successfully achieves real-time multimodal auditing, offline data privacy, and seamless dashboard synchronization. This design ensures scalable performance, data integrity, and zero ongoing cloud infrastructure costs for digital marketing agencies and content creators.

12. References

1. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

3. SQLite Official Documentation. (2026). *Write-Ahead Logging and Foreign Key Support*. `https://www.sqlite.org/doclist.html`

4. IEEE Recommended Practice for Architectural Description of Software-Intensive Systems (IEEE Std 1471-2000).

5. Ultralytics YOLOv8 Documentation. (2025). *Spatial Bounding Box Data Structures*. `https://docs.ultralytics.com/`

6. OpenAI Whisper Repository. (2024). *Robust Speech Recognition via Large-Scale Weak Supervision*. `https://github.com/openai/whisper`